

从早期到现代的主流网格生成算法综述

本文按时间顺序整理主要的二维/三维网格生成算法（包括三角形和四边形网格），并分析每种算法在用户提出的**六段式框架**（Input, Global Rule, Problem Reduction, Local Rule, Engine, Output）下的对应情况。如果算法符合该框架，则逐一对应框架模块；若不完全符合，则指出其偏离之处及原因。

1960s–1970s：结构化网格生成方法（映射和PDE法）

框架符合性：早期的结构化网格生成主要通过坐标变换或求解偏微分方程形成规则网格，**部分**符合框架。其过程没有明确的启发式局部规则，而是全局求解数学方程。

- **Input：**几何域的边界形状（通常为规则简单区域）以及期望的网格尺寸分布。典型输入是物理域与计算域的对对应关系。
- **Global Rule：**网格必须覆盖整个域且网格拓扑规整，无重叠元素，边界对齐。由于是结构化网格，还要求单元按规则阵列连接。
- **Problem Reduction：**将几何网格划分问题转换为数学映射问题。例如，Winslow提出通过求解Laplace方程来生成光滑的场作为内部网格坐标^①。Thompson等人1970年代发展了椭圆型PDE方法，在物理域与计算平面之间建立双调和映射^①。
- **Local Rule：**无显式的局部启发式规则。网格点位置由全局方程解确定，局部仅通过数值离散求解时的收敛条件控制光滑性。
- **Engine：**通过求解插值函数或Laplace方程生成内部节点坐标^①。引擎的objective是迭代求解PDE直至得到平滑的内部网格，method是数值迭代计算。整个过程中遵循几何边界条件和方程约束，而非启发式调整。
- **Output：**结构化的网格（二维为规则四边形栅格，三维为六面体网格），满足边界贴合且内部分布平滑均匀。输出网格通常质量较高，但由于完全遵循数学方程，缺乏针对复杂拓扑的灵活性。

偏离说明：该方法缺少独立的**Local Rule**模块，因为节点生成由全局PDE统一决定。另外未显式包含**Problem Reduction**步骤去处理复杂拓扑冲突——若输入几何过于复杂超出映射适用范围，则可能无解或需人工分块处理。

1980年代：非结构化网格生成三大方法兴起

1980年代被认为是非结构化网格划分技术的开创期^②。这一时期诞生了**前沿推进法**、**Delaunay三角剖分法**和**四叉/八叉树划分法**等关键算法^②。这些算法大多遵循几何有效性的全局规则，但在问题拆解和局部启发上有所不同。

1980 – 前沿推进法 (Advancing Front) 的提出

框架符合性：前沿推进算法总体符合框架思想，但**缺少显式的问题化简步骤**。它直接从边界开始构造网格，没有将问题映射为其他已解问题。

- **Input：**输入包括几何域的离散边界（边界节点和边划分）以及期望的单元尺寸分布。边界离散通常由用户提供或前处理生成。
- **Global Rule：**全局几何拓扑约束是：生成的单元逐步填充域内部，不得相互重叠，单元必须严格保边界而不越界。前沿推进还要求始终维护一个推进“前沿”界面，前沿两侧分别是已铺网格区和待划分区。

- **Problem Reduction**: 无明显的显式化简过程。该算法并未将问题转换为其他子问题，而是直接采用增量构造的策略。因此**Problem Reduction模块缺失**：算法假定输入条件允许填充整个区域，若遇到难以填充的“死腔”则可能失败，而没有预先化简或避免此情况的步骤。
- **Local Rule**: 有明确的局部规则。前沿推进每一步按照启发式规则在前沿上选取适当的边或顶点，构造新单元。例如，会根据邻近已生成单元的边长度和角度选择生成三角形的位置，避免过小角或单元退化。早期版本中，顶点位置可按规则微调以优化单元质量。
- **Engine**: 引擎以**迭代添加单元**为核心。objective是在保持网格有效的同时逐步推进前沿直至域内部填满。method为：从边界前沿开始，反复选取前沿上一段边，生成与之相邻的新三角形（或四边形），将该新单元加入网格并更新前沿³。算法持续进行，直到前沿在域中心合拢或无剩余空间。整个过程中遵循全局几何约束和局部构造规则。
- **Output**: 完整覆盖域的网格（通常是三角形单元）。前沿推进法以边界网格质量高著称，新单元逐层生成，边界附近单元质量极佳⁴；输出网格在前沿碰撞闭合处可能出现质量较差单元，但总体满足规定的尺寸分布和无缝覆盖域。

偏离说明：前沿推进没有显式的**Problem Reduction**模块——它不预先简化或拆解问题，而是“所见即所得”地直接铺设单元。这导致当域形状复杂（如前沿相遇形成复杂空腔）时，算法可能难以继续。此外，前沿法依赖局部启发控制新顶点位置，**Local Rule**占主导，缺乏全局优化步骤，这一点与框架中局部-全局兼顾理念有所不同。

注：Alan George在1971年首次提出将顶点创建与单元生成交替进行的思想，但这一想法一度被遗忘，1980年由Sadek重新发明了前沿推进算法⁵。因此实际最早的前沿推进应用可追溯到1980年前后。

1981 – Delaunay三角剖分算法 (Bowyer-Watson算法)

框架符合性：Delaunay三角剖分算法基本符合框架，但**没有单独的问题化简步骤**，属于直接求解型算法。其局部规则明确，即满足Delaunay空圆（空球）性。

- **Input**: 输入通常为一组离散的点（可来自边界取样或内部布点）以及可能的边界约束（要求某些边属于网格）。例如，Bowyer-Watson算法输入一系列平面点集；对带边界的情况需定义初始外部大三角形包围所有点。
- **Global Rule**: 全局规则是网格必须覆盖所有输入区域，且满足Delaunay性质：在平面三角剖分中，每个三角形的外接圆不包含任何其他顶点⁶。同时必须遵守基本拓扑规则：单元无重叠，邻接关系正确，已给定的边界约束边必须保留为网格边。
- **Problem Reduction**: 没有显式的problem reduction步骤。算法并未将构造Delaunay三角剖分转化为其他问题，而是直接利用已知的计算几何方法求解。所以**Problem Reduction缺失**：若输入点集不满足一般位置或约束有冲突，算法本身无法预处理简化，只能在构造过程中处理退化情况。
- **Local Rule**: 局部规则是Delaunay空圆准则（空圆性质）。具体体现为：当插入新点或调整网格时，任何三角形都必须满足其外接圆内部无其他点⁷。为保持这一局部规则，算法在插入点后会局部翻转边以消除不满足Delaunay性的三角形。这个局部操作（如Edge Flip）是算法的核心规则，保证所得剖分满足最大最小角性质（趋向于等角）。
- **Engine**: Bowyer-Watson引擎采用**增量插入法**。objective是将所有点都连接成Delaunay三角网。method：初始构造一个简单三角剖分，然后依次插入每个点：找到该点落入的三角形外接圆（或包含该点的当前三角形集合），删除这些三角形形成空腔，再将新点与空腔边界相连生成新三角⁷。在每次插入中，通过局部翻转确保Delaunay条件。该迭代过程一直持续到所有点插入完成⁸。整个过程中遵循全局拓扑有效性和Delaunay局部规则。
- **Output**: 输出为满足Delaunay性质的三角网格。其特点是总体高质量——没有“长细”的三角形（对于随机点分布常能避免极小角）⁹。若考虑边界约束，输出为约束Delaunay三角剖分（CDT），保留所有指定边界线段，同时保持局部的空圆条件。

偏离说明：Delaunay算法本身不执行**Problem Reduction**——假定输入点集问题良好即可完成剖分。如遇输入要求与Delaunay规则冲突（例如要求非凸边界），需要预处理或引入约束Delaunay算法解决¹⁰。此外，该算法的局部调整由数学性质决定，缺少针对特定目标（如单元梯度）的**Local Rule**调整，但这在框架上不是缺失，而是内置于Delaunay全局规则中。

1983 – 四叉树/八叉树网格划分法 (Quadtree/Octree Subdivision)

框架符合性：基于四叉树（2D）或八叉树（3D）的划分方法符合框架各部分。这类算法通过递归空间划分实现问题求解，可视作一种**问题化简策略**，将复杂区域逐步细分为简单子区域处理。

- **Input：**输入为几何域的边界描述以及目标细分大小控制参数（可为各区域期望单元尺寸）。例如，给定多边形区域及每处允许的最大单元尺寸。
- **Global Rule：**全局规则要求网格完全覆盖域且无重叠。由于输出单元源自规则划分，还需满足相邻区域细分层级相差不能过大（避免过大过小单元直接相邻）。对四边形/六面体输出时，要求单元正交排列对齐坐标轴或指定方向。
- **Problem Reduction：**采用递归空间剖分实现**问题化简**。算法将初始复杂域映射为包含它的简单正方形/立方体，然后递归细分：若某区域过大或包含复杂几何细节，则将正方形一分为四（四叉树）²；三维中将立方体一分为八（八叉树）。如此将原问题分解为许多规则的小块区域。这个分而治之过程持续，直到所有叶块满足尺寸和几何近似要求。最后再将这些块转化为网格单元。
- **Local Rule：**局部规则体现在细分判据和邻域平衡上。典型规则包括：**细分判据**——如果当前块内存在边界弧长过长或块尺寸大于局部允许大小，则细分；**平滑判据**——若相邻块细分级差超过一层，则细分较粗的块以平衡过渡¹¹。这些规则保证单元大小渐变合理，网格过渡平滑。
- **Engine：**引擎通过**递归细分与拼合**实现网格生成。objective是细分至所有块满足要求后，将块转换为网格。method：算法构造一棵树，从根（整个域）开始，根据Local Rule不断细分节点；细分在叶节点处停止，然后将每个叶节点（小块区域）转换成一个或多个网格单元。二维情况下，每个方形叶节点通常对角划分为2个三角形输出；如果目标是全四边形网格，则可直接使用正方形叶节点作为输出单元。三维情况下，立方体叶节点可划分为若干四面体或直接作为六面体单元（需特殊处理过渡区域）。整个引擎迭代直至无需进一步细分。
- **Output：**输出网格由大量规格化的单元组成——二维为近正方形的三角形或正交四边形，三维为立方体网格（带必要过渡单元）。这种方法产生的网格拓扑规整，元素大小渐变受控²。输出适合规则度要求高的应用，但在处理弧形边界时需要进一步加密边界或使用退化单元拟合。

偏离说明：四叉/八叉树算法严格遵循框架流程，没有明显缺失部分。不过，在**Local Rule**上它采用简单几何判据而非复杂启发；在**Engine**实现上，最终需要对不规则边界作额外调整（如加细或使用特殊多面体单元连接），这超出了基本框架描述但属于工程实现细节。

1990年代：质量优化与四边形/六面体网格生成

进入1990年代，网格生成算法在保证单元质量和处理复杂拓扑方面取得重要进展¹²。出现了**基于Delaunay细化的质量优化算法**、**全四边形网格生成算法**，以及**全六面体网格**的探索性算法等。这一时期算法更关注Problem Reduction和Local Rule的巧妙结合，以满足严格约束和优化目标。

1989 – Chew的Delaunay细化算法 (质量三角剖分)

框架符合性：Chew算法将Delaunay剖分与迭代加点结合，实现了对最小角的质量约束，符合框架各部分。其特色在于将原问题转化为**迭代细化问题**，即显式的问题化简过程。

- **Input：**输入为待剖分的多边形区域边界（无内部缝线）及初始离散点。可选还有一个最小角度阈值（如30°）。Chew算法通常从边界顶点开始，以此构造初始Delaunay三角网。

- **Global Rule**: 全局规则包括: 网格必须严格覆盖域且不侵犯边界, 所有原始边界约束边均保留; 同时全局需满足质量要求, 即**无角度低于30°的狭长三角形**¹³。Delaunay性质作为指导原则保持, 但Chew的第一次算法实际上避免了使用任何小于30°的角, 因此一定程度上融入全局硬约束。
- **Problem Reduction**: Chew算法将原始“不合格”网格逐步**细化**来解决质量问题。这是一种**问题缩减**: 初始问题 (三角剖分但质量不佳) 通过插入新点转化为更简单的新问题 (不良单元数减少的三角剖分), 不断迭代直到满足质量。具体而言, Chew提出任何瘦长三角形都可以通过在其外接圆插入新点来消除¹⁴。因此原问题被分解为一系列“消除瘦三角”的子问题。需要注意Chew算法为避免新点落在域外, 最初假设域边界没有小于90°的角, 以保证外接圆在域内¹⁵。
- **Local Rule**: 局部规则是**瘦长三角判据和加点法则**。每次迭代中: 查找存在角度小于阈值的三角形 (瘦三角形); 对于找到的瘦三角形, 在其外接圆中心插入新顶点, 前提是该圆不被域边界所截 (即不“越界”)¹⁵。如果外接圆跨出边界, 则改为将相应的边界边一分为二插入新点 (以免违反边约束)。这些局部规则确保每次操作都会提高最小角情况且不破坏Delaunay结构。
- **Engine**: 引擎为**迭代细化循环**。objective是消除所有不合格 (三角形角过尖) 的单元。method: 首先生成初始Delaunay剖分, 然后反复执行以下循环: 检测当前网格中是否存在瘦长三角形, 若有则按照Local Rule插入新点并更新Delaunay剖分网格¹⁴; 不断循环直至没有瘦三角形。算法保证此过程要么无限细化 (出现无解情况), 要么最终得到所有角均 $\geq 30^\circ$ 的网格¹⁴。Chew证明了在给定假设下算法会终止且生成满足质量要求的网格。
- **Output**: 输出为高质量的三角形网格, 各内角在30°至120°之间¹³。相较于未优化的Delaunay剖分, Chew网格的**最小角有严格下限**, 消除了细长元素。输出仍遵循Delaunay属性, 但为了质量可能略损失最严格的空圆性 (Chew第一次算法实际上放弃保留所有原始边界尖角, 以换取质量保证)。

偏离说明: Chew算法基本遵循框架, 但**Global Rule**中将“最小角阈值”提升为硬约束, 这在经典框架中属较偏好范畴。然而在该算法里, 最小角被当作必须满足的条件 (硬规矩) 来驱动算法设计¹³。此外, 其Problem Reduction通过迭代细化实现质量改进, 属于框架的有效扩展。

1995 – Ruppert的Delaunay细化算法 (实用质量剖分)

框架符合性: Ruppert算法是首个在实践中**真正令人满意的**保证质量网格算法¹⁶。它完全符合框架六部分——以更通用的Problem Reduction和Local Rule实现质量与尺寸最优网格¹²。

- **Input**: 输入为带内部边界的任意非凸多边形集 (即平面直线图, PSLG) 以及目标最小角阈值 (通常 $\approx 20^\circ$)。算法假设输入边界较“良性”, 比如没有过小角度, 否则可能需要预细化。
- **Global Rule**: 全局硬规矩包括: (1) 网格必须严格符合所有输入边界段, 任何输出三角形不得跨越或割裂原始边界¹⁷; (2) 输出网格各元素质量达标, 一般指最小角 $\geq$ 某阈值 (如 20°)。此外, 网格应尺寸渐进可控, 即不存在不必要的小元素 (Ruppert证明了其网格在满足质量的同时近似**尺寸最优**¹⁶)。Delaunay空圆性质作为隐含全局准则贯穿算法始终。
- **Problem Reduction**: Ruppert算法通过一系列判别和加点, 将复杂输入逐步**简化为可接受的局部配置**。首先, 它检测输入约束是否会导致无解 (如边界非常尖锐的角度违反质量阈值), 这相当于框架中的冲突检查; 然后算法采用**递归细化策略**: 把“生成高质量网格”这一复杂问题分解为反复**消除最差单元**的小任务, 直到没有坏单元¹⁴¹²。在此过程中, 如果出现约束边过长或造成坏单元, 则引入子问题——细分边界边。整体而言, Ruppert成功将网格质量问题转化为一系列**局部修补问题**, 每次修补简化了剩余问题的难度。
- **Local Rule**: 局部规则非常精细, 包含**加点规则**和**边界保护规则**两类。【局部规则A: 瘦三角细化】——如有三角形角度低于阈值, 取其外接圆为新点插入, 称为细化点。【局部规则B: 边界段细化 (防止侵入)】——如果细化点落在某条约束边的外接圆内 (称为该三角“侵入”了边界), 则**不直接插入圆心点**, 改为将该边界边中点插入 (称为分裂约束边), 以先消除侵入现象¹⁸。只有当没有任何侵入边界的坏三角时, 才插入坏三角的圆心点。以上规则确保: 不会因为插入点太靠近边界而破坏边界完整性¹⁸。同时, 任何插入都遵循Delaunay原则插入后即时保持三角形的空圆合法。
- **Engine**: 引擎为**迭代细化循环**, 类似Chew但更复杂。objective是消除所有不合格单元且保持所有边界完整。method: 先对输入边界进行初始化三角剖分 (可用Bowyer-Watson对所有边端点剖分并约束成CDT)。然后重复以下过程: 寻找当前网格中是否存在**坏三角形** (角过小) 或**过长的约束边** (相对邻近

三角过密度)。若有坏三角且其外接圆不侵入任何约束边,则在圆心插入点¹²;如果外接圆侵入了一条边,则改先细化该边(插入边中点)。每次插入点后,更新Delaunay剖分并继续。该循环持续,Ruppert证明了在一定条件下会终止并产出高质量网格¹⁶。整个引擎过程始终遵守Global Rule(不破坏边界,改善质量)和Local Rule(正确的插入次序和条件)。

- **Output:** 输出为高质量的三角形网格,既有严格的最小角度下限,又实现良好的尺寸渐进性¹⁶。Ruppert算法的网格通常被认为接近“最佳”——即在满足质量要求前提下,元素数量不会过度增加¹⁶。输出网格可处理带孔洞、非凸的复杂区域,质量在实践中令人满意¹⁶。

偏离说明: Ruppert算法几乎完美匹配框架,没有缺失部分。但值得注意的是,其Global Rule将“质量(角度)”与“尺寸最优”都提升为约束目标而非纯偏好¹²。这在框架中属于软偏好变硬约束的情况。不过Ruppert通过严格数学证明保障了这些目标的可满足,因此在此语境下质量和尺寸要求可被视作硬规矩的一部分。

1991 – Paving算法(全四边形前沿铺设法)

框架符合性: Paving是生成全四边形网格的经典算法¹⁹。它遵循框架流程,但在Local Rule和Engine上更复杂精巧,以解决四边形网格特有的奇偶匹配问题。整体上符合框架六部分。

- **Input:** 输入为待铺设区域的多边形边界(可凹可凸)以及目标单元尺寸分布。边界上一般要求节点间距已按尺寸场划分。初始时每条边界边作为一条“前沿”环。
- **Global Rule:** 全局约束包括: 最终必须得到**纯四边形**单元网格¹¹; 网格覆盖整个区域且与边界对齐; 内部节点每个应有偶数(一般为4)的相邻单元(奇数将形成不规则节点)。为了实现全四边形,必须成对地消除三角形的产生,这在全局上要求处理前沿碰撞时节点度数匹配¹¹。
- **Problem Reduction:** Paving通过**逐层铺设**将问题简化。它将填充复杂区域的全局问题转化为从边界向内反复铺设“层”(环状的四边形带)的子问题。每完成一层,问题规模缩小(待铺区域缩小)。算法还通过特殊处理,化解了**奇偶不匹配问题**: 当两股前沿即将碰撞时,如果边数奇偶不符,算法会在之前层中调整一行单元形状,使碰撞时边数相配²⁰。这种预调整相当于将原本棘手的奇偶冲突问题提前简化为可控的局部变形问题²⁰。
- **Local Rule:** 局部规则复杂且关键,包括: **铺设规则**——沿当前前沿添加一排新的四边形,使其一边连在前沿上; **节点规则**——当前沿转角处需要引入一个不规则节点(比如5度或3度节点)时,采用特定形状的小四边形组合来过渡; **碰撞处理**——两股前沿相遇时,若出现无法直接拼合的一余空隙,应用填充模板(例如临时生成一个三角形加以合并)确保最终输出仍是四边形。此外最重要的**奇偶调整规则**: 如果检测到两个前沿层即将以错误的奇偶性相遇,算法会在**尚未碰撞前的一层中扭曲单元排列**,以改变边数²⁰。这避免了最后一步出现单独的三角形单元,大大提高了算法成功率。
- **Engine:** 引擎以**分层前沿推进**为核心。objective是从边界向中心逐层铺满区域。method: 初始将区域边界视为第一道前沿,然后: 生成一圈贴着边界的四边形单元(即第一层); 更新前沿为这一层内侧的新多边形边界; 然后重复——在新的前沿再铺下一层四边形。如此往复,类似剥洋葱一圈圈填充,直到前沿在区域中心闭合。当前沿圈缩小时,若不同部分前沿互相碰撞,就按Local Rule处理碰撞和奇偶调整²⁰。算法在填充末期会出现少量特殊情况(如剩余区域呈五边形等),这时根据模板拼接剩余空隙,保证全四边形完成。整个过程由引擎调度,交替应用铺设新单元和局部调整,直至无空余区域。
- **Output:** 输出为覆盖域的全**四边形**网格¹⁹。除了必要的**不规则节点**(度数 $\neq 4$)外,网格由规则四边形组成,边界匹配输入。Paving生成的网格质量较高,特别是靠近边界处网格行列整齐; 遇到复杂形状时内部可能出现一些扭曲单元以消化奇偶差异²⁰。总体而言,输出满足工程实用要求且避免三角形单元。

偏离说明: Paving在框架中的Local Rule部分极为丰富,甚至比Engine本身更复杂——这是由于全四边形网格的拓扑限制,需要大量启发式规则支撑。不过它并未偏离框架,而是向其中注入了大量专有策略。在**Problem Reduction**方面,Paving通过逐层铺设将全局问题转化为了系列局部问题解决,这与框架思想一致。可以认为Paving完全适配框架,只是各模块实现细节复杂得多¹¹。

1995 – “Plastering” 算法 (全六面体网格的前沿充填)

框架符合性：“Plastering”（抹灰）是Sandia实验室提出的全六面体网格生成算法，基于前沿方法在三维中铺设六面体。由于全六面体情形非常复杂，Plastering **部分符合**框架：具有Input/Rule/Engine的基本结构，但**存在无法保证完成输出的问题**，即Engine可能中途失败，这是框架Output层面的缺失。

- **Input：**输入通常为三维几何域的封闭表面（已分割为全四边形网格的表面网格），作为六面体铺设的起始边界。此外需给定体积内目标单元尺寸或层数等控制参数。输入的表面网格节点度数对六面体充填有重要影响。
- **Global Rule：**全局约束是生成充满该体积的**全六面体单元**，内部不留空洞也不混入其他类型单元²¹。同时每个节点应满足六面体网格拓扑要求（通常期望内节点的度为valence= ~ 8 左右）。边界表面的网格固定，内部六面体必须与其兼容连接。全局上还要求逐层填充时不产生无法填充的奇异空腔。
- **Problem Reduction：**“Plastering”尝试将三维填充问题简化为**逐层铺设2D层片**的问题。基本思路是从表面开始生成一层六面体的壳，然后将问题缩小到剩余空间，再继续生成下一层。因此大的3D问题被分解为若干相继的“壳层填充”子问题。如果遇到两个生长前沿（壳层）相遇无法直接对接的情况，Plastering原始算法常会失败。后续提出的“Unconstrained Plastering”尝试通过**暂不立即划分壳层内部结构**的策略，将难以填充的局部冲突延后处理，以提高成功率²¹。这相当于一种问题化简：先铺设六面体片但不立即决定其内部网格划分，让多个片在空间相遇后再整体考虑划分，从而避免早期决策造成死锁²¹。
- **Local Rule：**局部规则包括：**前沿延拓规则**——在当前六面体层的前沿表面上选择一四边形面，向内复制出一个新的六面体单元；**节点配额规则**——控制当多个六面体前沿在某一点汇合时，每个节点处不能有奇异过多或过少单元，这需要在铺设过程中跟踪每个前沿的汇合情况，并可能调整六面体形状以满足节点6面度要求；**碰撞填充规则**——当两个推进的六面体前沿形成一个尚未填充的小空腔时，采用特定模板（如5面体模板、锥形模板）将该空腔用最后一个六面体填满。由于六面体要求三维配对，这些规则远比二维复杂且经验性强。
- **Engine：**引擎采用**分层前沿推进**（三维）。objective是在整个体积内逐步充满六面体。method：从输入表面网格作为初始前沿，向内拓展生成一层贴着表面的六面体“薄壳”。当一层完成后，将前沿推进到下一层，即内侧新形成的自由表面，再继续生成六面体。如此层层推进。如果不同方向推进的层在内部相遇，Plastering会尝试按照Local Rule调整最后几个单元以拼合。如果严格的Plastering算法无法填充某区域（即出现奇异空腔无模板可用），则算法在此中止并报失败²¹。因此Engine可能在未达到目标时结束，不能保证一定完成全域充填。这也是全六面体自动生成的难点所在²²。在“无约束抹灰”改进中，引擎允许前沿壳碰撞时暂时不细分，待碰撞完成后再统一划分壳层，以减少失败几率²¹。
- **Output：**理想情况下，输出为覆盖整个体积的**全六面体网格**，无其他类型单元²¹。输出网格与输入表面网格完全一致且光滑过渡。如果算法在某些复杂区域无法继续，则可能没有输出（或输出部分区域网格），需要人工干预调整几何或分块后再网格生成²¹。总的来说，在适当条件下Plastering可生成高质量的六面体网格，但对复杂模型尚无万能保证²²。

偏离说明：Plastering算法**未完全符合框架**，主要问题在**Output环节的保障不足**。与框架假定最终能输出解集不同，Plastering在遇到复杂拓扑情况时可能**无法产出合格输出**（即失败），说明其Engine缺少完备性。这体现为未能彻底解决六面体前沿碰撞时的**Problem Reduction**——尽管Unconstrained版本尝试推迟决策，但全局仍可能有无解配置²¹。因此，全六面体自动剖分被认为“目前尚无可靠通用算法”²³。这一偏离反映出六面体网格生成的难度而非框架不适用性：框架提供了分析视角，但现实中Engine可能陷入无解局面，框架的Output模块因此缺失（无有效输出）。

1995 – 气泡填充法 (Bubble Mesh, Shimada 等)

框架符合性：气泡算法将网格节点当作具有相互排斥力的“气泡”，通过物理模拟均匀布点，然后再构建Delaunay网格²⁴。该算法**基本符合**框架，各模块对应明确：它把问题转化为物理平衡问题（Problem Reduction），通过局部力规则驱动迭代（Local Rule），最后用Delaunay三角化生成网格。

- **Input：**输入为待划分区域的边界以及目标元素尺寸（通常通过期望节点密度或“气泡”半径给定）。可以是平面区域或三维体域。初始时，在区域内布置一定数量的“气泡”（节点），可随机或规则初始。
- **Global Rule：**全局要求为：最终气泡（节点）分布应填满区域且彼此距离符合给定半径（密度）。任何两气泡不得靠得过近（小于一定间距），相当于硬性规定了节点间最小距离²⁵。同时所有气泡应尽量均匀分布，无明显聚堆或过大孔隙，即满足整体的均匀“蓝噪声”特性。这均属于全局的几何/分布规则。
- **Problem Reduction：**将几何网格问题转化为**动力学平衡问题**。Shimada等人观察到，紧密堆积的圆（球）中心连接可形成Voronoi图，从而生成网格²⁴。因此问题被简化为：在区域内放置一系列半径适当的球，使其互相不重叠又尽可能紧密填充²⁴。这是一个物理模拟问题，通过调整气泡运动达到稳态就解决了点生成问题。随后再用标准Delaunay三角剖分连接这些点得到网格²⁶。可见，该算法核心是先把**网格节点分布**作为独立子问题求解。
- **Local Rule：**局部规则以**相互作用力**形式出现。每对气泡之间存在排斥力：当两气泡距离小于理想值时，它们受力相互推开；距离过大则力为零或有轻微吸引（部分实现可能加入轻微吸引确保凝聚）。此外靠近边界的气泡可能受边界力推动回域内或贴附边界。气泡还具有**碰撞规则**：禁止气泡重叠，一旦接近触碰会产生强斥力使其分开²⁷。这些局部力学规则指导每个气泡逐步移动到均衡位置，类似模拟粒子系统。
- **Engine：**引擎采用**迭代物理模拟**。objective是所有气泡达到平衡（净力接近零）且分布均匀覆盖域。method：初始化若干气泡后，反复执行小时间步，对每个气泡计算合力矢量（根据Local Rule来自邻近气泡的斥力和边界力），然后移动气泡略微位置；同时可能根据需要增减气泡数量以调整密度（某些实现中逐步增加气泡充满区域）。这一过程持续迭代，类似“分子动力学”模拟，直到气泡运动趋于停止状态，即任意气泡与邻居距离均接近期望值。达到平衡后，所有气泡中心作为节点，以Delaunay方法连接成三角形（或四面体）网格²⁸。引擎过程严格遵守全局不重叠规则，借助局部物理规则收敛。
- **Output：**输出为高质量网格（通常为三角形网格，如果在体内则是四面体网格）。由于节点分布接近均匀且符合目标尺寸，输出网格的形状质量较好，没有过小角或过大面积差异²⁴。边界上节点也基本均匀贴合。可以说，Bubble Mesh方法输出的网格质量很大程度取决于粒子平衡配置的质量——这一配置实际上等价于**Centroidal Voronoi Tessellation**的一种近似，因此输出网格接近各向同性优化的理想状态。

偏离说明：气泡法并未明显偏离框架，但它的**Engine**和**Local Rule**并非传统离散算法而是连续模拟。这意味着收敛速度和精度可能无法严格保证。不过从框架角度，Bubble Mesh最大特点在于**Problem Reduction**：它将组合离散问题转化为了物理连续问题求解，这一步在传统框架中较少见，但本质仍是满足全局规则的另一途径²⁴。因此框架依然适用，只是实现方式不同。

1999 – Q-Morph算法 (三角网格转四边形网格)

框架符合性：Q-Morph是间接生成全四边形网格的方法，先生成三角网格再通过一系列局部变换合并为四边形²⁹。它符合框架：有清晰的问题化简（借助三角网格）、局部规则（几何变换规则）和引擎流程。

- **Input：**输入为多边形区域或曲面，其上已有高质量三角形网格（通常由Delaunay或前沿法生成）。还需提供尺寸分布信息（隐含在初始三角网格中）和边界条件。输入三角网格应较均匀且无过小角，以利于后续转化。
- **Global Rule：**全局要求输出**纯四边形**网格，且拓扑一致覆盖区域，没有残留三角形单元²⁹。同时边界条件要求输出网格的边必须吻合原边界网格（即如果原三角网格边界由若干短边组成，输出四边形边界需与之对齐）。另外全局上希望四边形“行列”尽量排列有序，与边界平行³⁰并减少不规则节点数。

- **Problem Reduction**: Q-Morph的关键思想是**将全四边形网格问题简化为已解决的三角网格问题**。它先对区域生成三角形网格（这一步利用了成熟的三角剖分算法，等于将原问题映射到三角网格领域）³¹。接着算法把**合并三角形得到四边形**作为子问题处理：这是通过一系列局部操作完成的，不需要全局重新布局。因此，大问题被拆解为两个阶段——先三角剖分，后局部合并。第二阶段进一步简化为沿边界推进的合并子问题：算法从边界开始，逐步将相邻三角形配对转换成四边形，从而将问题层层缩小²⁹。
- **Local Rule**: 局部规则是一组**三角形到四边形的拓扑变换规则**。例如：**三角-三角合并**——两个共享一条边的相邻三角形可以合并为一个四边形，此为主要操作²⁹；**旋转优化**（边翻转）——在某些情况下，当不能直接合并时，算法会对三个三角形进行边交换（旋转），重新排列成两个可合并的四边形；**填充模板**——在边界或尖角处，如出现单个三角形无法匹配，使用特殊模板将其与邻近单元合并（例如引入一个临时五边形然后拆分）。Q-Morph还定义了**优先规则**：通常优先处理靠近边界的三角形，使前沿逐步内缩，同时避免留下孤立三角形²⁹。这些规则保证局部转换操作既遵守几何合理性又服务于全局目标（尽量平行于边界生成“四边形层”）。
- **Engine**: 引擎工作流程分两阶段。第一阶段已由输入提供（初始三角网格生成）。第二阶段执行**迭代三角合并**。objective是将所有三角形消除，得到全由四边形组成的网格。method：从区域边界开始，顺着边界“前沿”遍历：检查每对相邻三角形是否能直接合并为四边形，如可以则合并并将该四边形纳入输出；若某处三角形无法简单配对（例如局部有奇数个三角形），则应用局部旋转等变换，使其凑成可配对情形再合并²⁹。算法沿着前沿推进一圈后，新的前沿向内推进一层（类似铺设层概念），再重复合并过程。如此逐层推进至中心。如果最后剩下极少数未配对三角形，使用预定模板进行处理，确保所有三角形消除。引擎每步均遵循Local Rule，合并后即时调整拓扑，直到完成。
- **Output**: 输出为区域的全**四边形网格**²⁹。输出质量取决于初始三角网格质量：通常如果初始三角形分布良好，Q-Morph输出的四边形也排列规整³²。该算法力求使生成的四边形“行”与边界平行，内部不规则点较少³²。最终输出满足工程对全四边形网格的要求，且因由三角形转化，单元质量一般也较高。

偏离说明：Q-Morph基本遵循框架流程，没有缺失模块。但相对于直接全四边形算法（如Paving），它借助**Problem Reduction**大大简化了任务：先解决三角网格，再做局部转换²⁹。因此框架中Problem Reduction在此扮演重要角色，使Engine主要处理简单局部操作而非全局规划。这体现了框架灵活性，即允许算法通过引入中间问题（三角网格）来间接实现目标。

2000年代：优化型与混合型网格生成新方法

进入21世纪，网格生成在优化理论和混合算法方面继续发展。突出特点是**基于能量优化的点布置方法、各向异性网格算法、全局参数化方法**等开始应用于网格划分。与此同时，生成全四边形/全六面体网格的新路劲也在探索（如基于场引导的划分）。这些新方法通常强调**软偏好目标**（如单元质量最佳、各向同性/各向异性控制）和**全局-局部迭代优化**，符合框架但强化了优化组件。

2000 – 重心Voronoi划分 (CVT网格, Lloyd优化法)

框架符合性：基于CVT（Centroidal Voronoi Tessellation）的网格生成通过迭代优化点的位置，追求整体网格质量最优。它符合框架各部分，但**不以硬约束驱动而是以软目标优化为主**——这意味着Global Rule侧重有效覆盖，而质量提升更多作为优化目标实现。

- **Input**: 输入为几何域边界、初始采样点集（可随机或规则布点），以及期望的点密度分布（或等价的目标单元大小函数）。初始点集数量通常由用户或算法根据面积体积估计给出。
- **Global Rule**: 全局硬规则是：所有点必须留在域内且大体覆盖域的每个部分，没有大的空白区域；输出三角网格需无重叠并严格拟合边界。没有特定角度或形状硬约束要求——质量要求作为软偏好融入目标函数。
- **Problem Reduction**: 算法将网格优化问题转化为**能量最小化问题**。具体来说，采用Lloyd迭代（Voronoi松弛）使点集对域的覆盖均匀化³³³⁴。其等价于最小化Voronoi单元的二次距离和（量化误差），这是明确的连续优化问题。这一步骤简化了原问题：与其直接构造高质量网格，不如先找到最

佳点分布。一旦点的位置优化好，再用Delaunay三角剖分这些点即得到高质量网格（因CVT点分布对应非常好的三角形质量）。因此Problem Reduction体现在：将“生成高质量网格”拆解为“优化点位置+标准三角剖分”两个步骤，其中优化点位置是通过能量函数求解的连续问题。

- **Local Rule:** 局部规则为**迭代重定位**：每个点在迭代中移动到了其Voronoi多边形的质心位置¹³。Lloyd算法的规则是同步执行：计算全局Voronoi图（全局步骤），然后对每个区域，将对应的采样点移动到该区域的质心（局部步骤）³³。这样每轮迭代后点集分布更均匀。该局部更新遵循的规则可以理解为：**点尽量位于各自Voronoi区域的中心**，从而均衡邻近关系。没有其他启发式，纯粹依据当前全局状态计算出的局部优化方向。
- **Engine:** 引擎采用**局部-全局交替优化迭代**。objective是最小化量化误差（或等价，使点分布均匀），达到近似CVT状态¹³。method：先根据当前点计算Voronoi划分（全局更新Voronoi结构）；然后将每个点移动到了其Voronoi区质心（局部并行更新）；重复该过程。迭代单调减少总体能量并最终收敛¹⁴。此过程中无硬约束违背——若无边界，点会无限趋于均匀分布；有边界则可将边界点固定或用惩罚法处理。整个算法遵守Global Rule（点不出边界，覆盖域）并按照Local Rule更新。通常引擎迭代若干轮后达到稳定分布。
- **Output:** 输出为经过优化点集的Delaunay三角网格。由于点分布趋于各向同性均匀，其Delaunay三角剖分质量极佳：不会有极小角三角形，且元素大小分布均匀平滑。这满足高质量网格的要求，虽然算法本身没有直接约束角度，但通过优化自然达到了优化目的¹³。输出网格还具有蓝噪声特性，对后续仿真非常有利。

偏离说明：CVT/Lloyd算法严格遵循框架，只是**Global Rule**中质量要求不是硬约束而是通过**软偏好**逐渐实现³³。换言之，在框架角度它更强调软目标优化（均匀性）而硬规矩仅限于覆盖域和无重叠等基本条件。这与框架思想并不冲突，而是实现路径不同：通过连续优化实现局部-全局平衡³³。需要注意的是，CVT方法对**Problem Reduction**的运用非常巧妙，即分离出点分布优化，这一思路后来广泛影响了各类网格与采样算法。

2004 – DistMesh算法 (基于距离场的物理拉伸法)

框架符合性：DistMesh是一种简洁的网格生成方法，将节点视为相连弹簧构成的力学系统，通过松弛达到均衡³⁵。它与Bubble方法类似，本质符合框架：利用Local Rule（弹簧力）和Engine迭代实现全局满足规则，但没有显式复杂的**Problem Reduction**，而是直接处理。

- **Input:** 输入为几何域的**距离函数**表示（可计算给定点到边界的距离，用正负表示内外），初始节点分布（通常为在域内均匀分布的一套点，可规则网格或随机点）以及目标边长度函数（决定弹簧自然长度）。
- **Global Rule:** 全局硬规矩：节点须留在域内部（依靠距离函数确保）且尽量均匀覆盖域，无堆积或大空隙；网格连接拓扑通常选用Delaunay连接（算法迭代过程中不断重建Delaunay三角网来更新连接）。在物理模拟意义上，没有其他硬约束。
- **Problem Reduction:** DistMesh并未使用显式的问题化简步骤（**缺少单独的Problem Reduction模块**）。它直接在初始节点集上施加物理模拟，使其趋于理想分布，然后输出网格。因此，与Bubble法先计算Voronoi不同，它跳过了明确的数学问题转换，而是直接把网格问题当作力学平衡问题处理。不过，可以认为**将网格质量问题隐含转化为了弹簧系统能量最小化问题**，这相当于一种隐式的问题简化：利用物理能量函数代替显式几何标准。
- **Local Rule:** 局部规则由**弹簧力模型**给出。节点间用弹簧相连（通常依据当前三角剖分的边连接），每根弹簧有期望长度（由目标尺寸函数决定）。规则是：每对连接节点之间产生作用力——距离大于理想长度则相互拉近，距离过小则相互推开³⁵。同时每个节点根据距离函数检查是否越出边界，如越界则被投影回边界表面上。所有节点同时受力调整，形成一种“质点弹簧系统”的局部更新规则。
- **Engine:** 引擎通过**迭代松弛**求解平衡。objective是所有弹簧近似达到自然长度，系统能量最低。method：循环执行以下步骤：计算当前网格的Delaunay三角剖分以确定节点连接关系；针对每条边计算弹簧力矢量并累加到相关节点上；然后将每个节点沿合力方向移动一小步（通常使用阻尼以避免震荡）；将移动后的节点投影回域内（或边界）；重复迭代。随着迭代，节点分布逐渐逼近理想，移动幅度越来越小。当最大节点移动小于阈值时停止。整个过程中，Global Rule（节点不出域、覆盖合理）由距离函数投影和弹簧网络共同保障，Local Rule则在每一步都作用于节点。

- **Output:** 输出为以最终节点为顶点的Delaunay三角形网格。由于节点通过弹簧相互作用趋于均匀，输出网格质量较高——单元形状接近均匀，过大过小单元较少，且边长接近目标函数要求³⁵。DistMesh产出的网格虽非严格质量最优，但通常满足工程需要，且算法实现简洁。输出自动满足边界条件，因为所有出界节点被拉回边界，使边界拟合较精准。

偏离说明：DistMesh严格来说未明确区分**Problem Reduction**，属于直接数值迭代法，这一点偏离框架中将问题映射/分解的步骤。不过可以视其为将问题自然融入Engine处理，而非完全忽略。除了这一点外，其他模块如Local Rule（弹簧规则）、Engine迭代和Global规则都很清晰，符合框架思想。DistMesh的优势在于实现简单统一，用物理直觉取代复杂几何规则，这是框架方法的另一种体现。

2009 – 混合整数四边形分割 (MIQ, 全局参数化法)

框架符合性: MIQ (Mixed-Integer Quadrangulation) 是一类利用全局场（方向场、参数化）生成四边形网格的算法²⁹。它通过求解全局最优的参数化来铺设四边形，非常注重**Global Rule**与**Problem Reduction**：将网格对齐等要求转化为场优化问题。整体符合框架，但部分实现中Problem Reduction和Engine高度融合，属于以数学优化替代启发的方式。

- **Input:** 输入通常为一个曲面三角网格（表示几何形状），以及用户期望的对齐方向（例如主曲率方向或手工指定方向场）。还需提供分割尺寸，或通过设置场的缩放来隐含控制单元大小。初始还可能某些约束奇异点位置等。
- **Global Rule:** 全局硬规则包括：输出网格为**无缝的全四边形**划分；四边形网格线应当尽可能**对齐**输入给定的方向场（例如与表面主方向一致），实现流线效果；整个曲面的参数化无折叠、无空隙，保证拓扑正确地覆盖曲面。特别地，MIQ要求方向场在整个域上是连续或仅在离散对称处跳变90度，这是一项全局条件，确保输出网格连贯³⁶。
- **Problem Reduction:** 核心在于将**离散网格布局问题转化为连续方向场和坐标场求解问题**。MIQ首先求解一个光滑的四向量场（cross field）使其遵循给定引导方向，同时需要在网格奇点处设置奇异性。然后将该场**积分**为一个全局参数坐标网格。这本质是**连续优化问题**：通过混合整数优化求解场，使其最接近平滑且满足边界条件和对齐要求³⁶。这个过程显著化简了问题：原本要一步步铺网格，现在通过求解一个偏微分方程/优化问题一次性给出整个网格的结构。可见Problem Reduction在此将几何离散问题转换为了一个带整数约束的连续优化（混合整数规划）问题。
- **Local Rule:** 此类算法的Local Rule体现在**局部场调整和整数约束**上。例如，在Mixed-Integer优化中，会将方向场在每个三角形上离散采样，并要求相邻处场方向差为0或90度的整数倍，以保证积分后形成网格线连续³⁶。局部而言，每个三角形的方向变量会被“四舍五入”到最近的合法方向（0°, 90°, 180°, 270°），这是一种局部规则。另一局部规则是当场具有奇异源时，在其邻域布置一个适当valence的不规则点，以**消化场的旋转总量**。这些局部约束在优化中通常转为线性同余条件。因此虽然算法由全局优化主导，但内部隐含许多局部规则（如网格奇点的指数、场的平滑限制等）。
- **Engine:** Engine分两步：**先场优化，后参数化生成**。objective是找到能生成良好四边形的参数化网格。method：首先全局求解方向场/参数化问题（通常构建一个能量函数，比如偏离引导方向的平方和，加入整数约束后用优化器求极小）。求解得到的是一个网格的UV坐标映射或网格流线结构。然后据此提取四边形：沿参数等值线划分曲面得到网格。整个引擎过程以一次（或数次）全局求解完成，无需迭代铺设。若方向场求解有迭代，也是在连续空间迭代，不涉及逐元素增加。Engine遵循Global Rule，因为全局求解已经将约束内置，Local Rule由优化器满足整数条件来保证。最终网格直接从连续结果读出。
- **Output:** 输出为表面上的**全四边形网格**，其网格纹理与输入方向对齐良好，具有美观的流向结构³⁶。单元尺寸可以通过参数刻画均匀或渐变。由于采用全局优化，输出网格质量在对齐和均匀度上都很高，各单元形状畸变小。当然，不规则节点的位置和数量取决于场奇异点配置，一般尽可能少且合理布局。总体而言，MIQ类方法能自动产生布局合理的四边形网格，在CAD和图形领域被认为是现代高质量方法之一。

偏离说明：MIQ算法充分运用了框架思维，只是**实现方式偏向数学优化**而非传统启发式。因此**Engine**表现为求解线性系统或混合整数系统，而非逐步构造，这一点在框架描述中依然可以涵盖（Engine.method换成“全局优化计算”）。需要注意的是，MIQ非常强调**Global Rule**（方向对齐全局一致性）在先，牺牲了一部分Local

Rule的简单性——其局部规则都融入优化约束了，使人不易直接看出。但从宏观看，它是满足框架要求的：有明确Input、全局约束、通过转换为参数化问题（Problem Reduction）解决，再输出结果。偏离可能在于Local Rule弱显式、Engine一次性完成，不过这属于实现差异，不违背框架。

2015 – Instant Meshes即时四边形网格 (场引导的贪婪算法)

框架符合性：“Instant Meshes”是一种实用工具，用于快速产生场引导的四边形网格。它首先计算指导场，然后用启发式贪婪策略铺设网格，因此**部分符合**框架：Global Rule（场方向）由优化给出，Engine用局部贪婪合并线。相较MIQ，其Engine不是全局优化而是近似算法。

- **Input：**输入为曲面（三角网格表示）和方向场（通常自动计算，也可由主曲率方向生成），以及目标面元大小。可选输入还包括用户指定的一些引导线或奇异点。
- **Global Rule：**要求输出**各向对齐**的四边形网格：网格边缘跟随输入方向场，并保证整个曲面无缝覆盖。另一个规则是**快**，因为工具强调交互速度，所以并未将高精度最优化作为目标，而是快速产生较优解为主。这种时间要求不属于几何框架约束，但影响了实现策略。
- **Problem Reduction：**首先通过求解简化的场（如梯度场）得到指导方向并生成流线，这一步类似降低了原网格布局问题的复杂度。Instant Meshes通过解Poisson方程得到一个可以直接积分的方向场，相比MIQ降低了整数约束（采用流场追踪结合周期调整替代混合整数求解）。这相当于将**复杂优化问题近似化**，使后续引擎更简单快速。因此Problem Reduction在此是**将严格的场优化问题转换为可贪婪追踪的场**。
- **Local Rule：**Engine主要由局部贪婪规则驱动：**流线合并规则**——沿场方向从各种种子点出发追踪线条，当两条流线接近且符合网格拓扑要求时，连接形成四边形面；**冲突消解规则**——当流线交叉或过近，按照一定优先级舍弃或调整部分线段，保证最终网格无瑕疵地覆盖表面。这些规则逐步在局部构建四边形单元，同时参考Global方向场以不偏离大方向。
- **Engine：**Engine按**贪婪迭代**构造网格。objective是在极短时间内生成对齐合理的四边形网格。method：根据方向场，首先生成一系列初步的粗网格线框，然后局部调整这些线框节点对齐到最近的方向场整齐角度，接着贪婪地连接节点成网格面，过程中如发现拓扑错误则局部修改或添加不规则点。这个过程不像优化那样有全局收敛保障，但在实践中很快产出结果。Engine遵循Global Rule因为时刻以方向场为引导，Local Rule确保每一步连接都是合法且尽量优化局部形状。
- **Output：**输出为一个各向一致的四边形网格，通常在几何细节较低频区域能保持与场对齐（如纹理状分布），在高曲率区域可能出现一些不规则以适应曲率。质量上，比起MIQ等全局优化方法稍逊一些（比如有些单元会略变形），但仍满足大多数应用的要求，而且**生成速度极快**。输出的奇异点位置和数量可能不完全最优，但以可接受方式分布，从而完成网格全覆盖。

偏离说明：Instant Meshes在框架的**Engine**实现上偏离了追求全局最优的思路，转为贪婪启发式，因此**Local Rule**在其中占比较大而**Problem Reduction**求解偏弱。这导致输出虽符合硬约束但未必达到最高软目标。不过从框架看，它仍提供Input（曲面+方向）、Global Rule（方向场引导）、Problem Reduction（方向场近似求解）、Local Rule（贪婪拼合）、Engine（流线追踪生成）、Output（四边形网格）这些环节。只是在精确性上不是最优，这是算法权衡速度的选择，不算框架缺失。

2010年代：框架驱动的新探索（场引导六面体、机器学习等）

2010年代出现了更多新思路，例如**框架场引导的全六面体网格**（通过在体内建立三维框架场，类似三维的MIQ）、**多块结构自动划分**、以及尝试用**机器学习**辅助网格剖分等。这些前沿方法大多仍可用本文框架分析，但由于技术尚在发展，其Engine可靠性和框架完整性有待提高。例如：

- **全六面体框架场法：**通过计算体积内的三维正交场，并优化形成网格的“骨架”，再充填六面体。这类似全局Problem Reduction（场优化）+局部填充Engine。但目前实现复杂，对一般域仍无完全保证，仅在一定规则几何上成功。框架分析可发现，其难点在于Global Rule（场需满足复杂拓扑）和Engine（六面体充填）衔接仍未完全解决。

- **多块分解法**：自动将复杂几何分解为可映射的规则块，然后对每块生成结构化六面体网格。这个思路显然符合框架的Problem Reduction（几何分块）和Engine（每块映射铺网格），但自动分块本身非常困难，需要智能算法或启发。当前多为半自动，人为指导分块。
- **机器学习辅助**：利用深度学习根据大量样本预测网格划分的某些步骤，例如点布置或局部拓扑修补。其作用相当于一个经验性Local Rule模块，由训练数据决定规则。ML方法通常嵌入在传统算法流水线中，并不改变框架结构，只是用数据驱动的方式实现部分Engine或Local决策。

综上，**主流网格生成算法**从早期结构化方法到近代智能优化，各自基本要素都可映射到统一的几何求解框架中。尽管实现策略多样（经典算法偏启发式，现代算法偏优化和混合），但分析其Input/Rule/Engine/Output，有助于理解算法设计思路与局限性。这种框架视角也为将来改进网格算法、融合多种方法提供了清晰指引。

1 3 22 36 Mesh generation - Wikipedia

https://en.wikipedia.org/wiki/Mesh_generation

2 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 23 33 34 people.eecs.berkeley.edu

<https://people.eecs.berkeley.edu/~jrs/papers/umg.pdf>

24 25 26 28 A Clustering-Based Bubble Method for Generating High-Quality Tetrahedral Meshes of Geological Models

<https://www.mdpi.com/2076-3417/10/15/5292>

27 [PDF] Agent-based Algorithm for Spatial Distribution of Objects - KAUST ...

<https://repository.kaust.edu.sa/bitstreams/fd00fb6c-3608-4f0d-ba99-811aad7d56b0/download>

29 [PDF] All-quad meshing without cleanup - Ahmed Mahmoud

https://ahdhn.github.io/files/all_quad_paper.pdf

30 32 q-morph: an indirect approach to advancing front quad meshing

<https://www.semanticscholar.org/paper/Q-MORPH%3A-AN-INDIRECT-APPROACH-TO-ADVANCING-FRONT-S.J.OWEN-Staten/6d340e49ce348897d1898845bc01f27465731df4>

31 [PDF] Q-Morph: an indirect approach to advancing front quad meshing

https://pages.cs.wisc.edu/~csverma/CS899_09/qmorph.pdf

35 [PDF] A Review on Mesh Generation Algorithms - IJRAME

<http://www.ijrame.com/wp-content/uploads/2019/03/V5i503.pdf>